

Métodos Numéricos - Interpolación

Ricardo Largaespada

7 de noviembre de 2018

1. Método de Lagrange

La forma más simple de una interpolación es un polinomio. Siempre es posible construir un polinomio $P_{n-1}(x)$ de grado $n - 1$ que pase por n puntos distintos.

Una manera de obtener este polinomio es la *fórmula de Lagrange*

$$P_{n-1}(x) = \sum_{i=1}^n y_i l_i(x)$$

donde

$$\begin{aligned} l_i(x) &= \frac{x - x_1}{x_i - x_1} \cdot \frac{x - x_2}{x_i - x_2} \cdots \frac{x - x_{i-1}}{x_i - x_{i-1}} \cdot \frac{x - x_{i+1}}{x_i - x_{i+1}} \cdots \frac{x - x_n}{x_i - x_n} \\ &= \prod_{j=1, j \neq i}^n \frac{x - x_j}{x_i - x_j}, \quad i = 1, 2, \dots, n \end{aligned}$$

son llamadas las *funciones cardinales*. Por ejemplo, si $n = 2$, el polinomio es la línea recta $P_1(x) = y_1 l_1(x) + y_2 l_2(x)$, donde

$$l_1(x) = \frac{x - x_2}{x_1 - x_2} \quad \frac{x - x_1}{x_2 - x_1}$$

Con $n = 3$, la interpolación es parabólica: $y_1 l_1(x) + y_2 l_2(x) + y_3 l_3(x)$. Donde ahora

$$\begin{aligned} l_1(x) &= \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)} \\ l_2(x) &= \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)} \\ l_3(x) &= \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)} \end{aligned}$$

Las funciones cardinales son polinomios de grado $n - 1$ y tienen la propiedad

$$l_i(x_j) = \begin{cases} 0 & \text{si } i \neq j \\ 1 & \text{si } i = j \end{cases} = \delta_{ij}$$

donde δ_{ij} es el delta de Kronecker. Para probar que el polinomio interpolante pasa a través de los puntos, podemos sustituir $x = x_j$, el resultado es:

$$P_{n-1}(x_j) = \sum_{i=1}^n y_i l_i(x_j) = \sum_{i=1}^n y_i \delta_{ij} = y_j$$

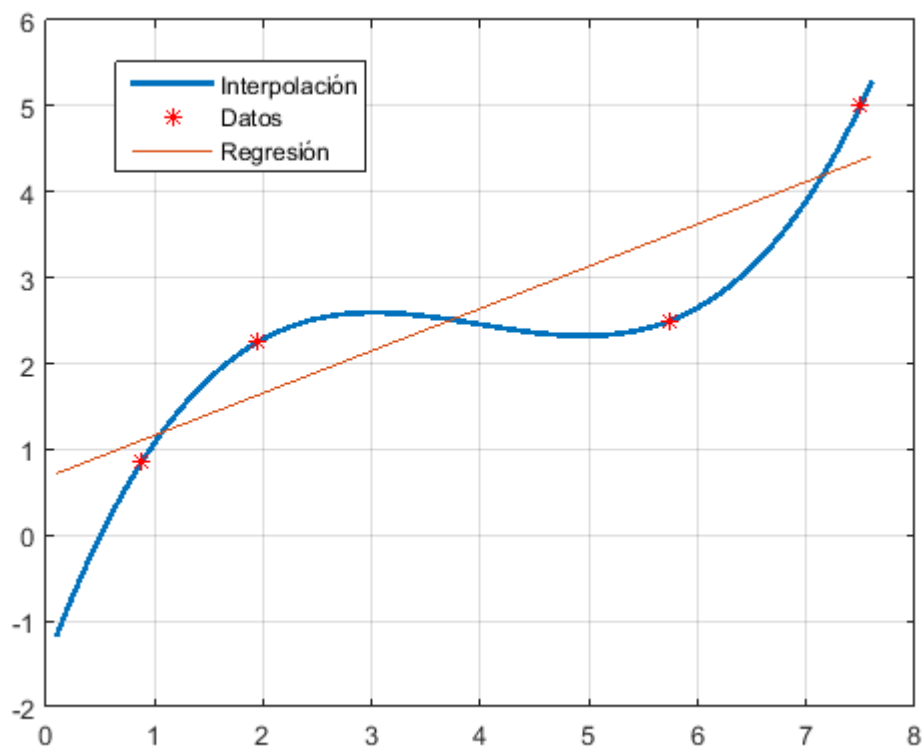


Figura 1: Interpolación y ajuste de curva de datos.

Puede demostrarse que el error en la interpolación polinómica es

$$f(x) - P_{n-1}(x) = \frac{(x - x_1)(x - x_2) \cdots (x - x_n)}{n!} f^{(n)}(\varepsilon)$$

donde ε está en algún lugar del intervalo (x_1, x_n) su valor es de otra manera desconocido. Es instructivo observar que cuanto más lejos se encuentre un punto de datos de x , más contribuirá al error en x .

Ejemplo 1.1. Dados los datos

x	1.6	2	2.5	3.2	4	4.5
$f(x)$	2	8	14	15	8	2

- Calcule $f(2.8)$ con el uso de polinomios de interpolación de grados 1 a 3. Elija la secuencia de puntos más apropiada para sus posibles predicciones.
- Grafique los polinomios ajustados.

Si el grado del polinomio es 1, seleccionamos a 2 puntos para la interpolación, digamos $(2.5, 14)$ y $(3.2, 15)$; de esta manera el Polinomio ajustado según la fórmula de Lagrange es:

$$P_1(x) = \frac{x - 3.2}{2.5 - 3.2} \cdot 14 + \frac{x - 2.5}{3.2 - 2.5} \cdot 15 = \frac{10x}{7} + \frac{73}{7}$$

Entonces $f(x) \approx P_1(x) = \frac{10x}{7} + \frac{73}{7}$. Por tanto, $f(2.8) = 101/7 = 14.428571429$.

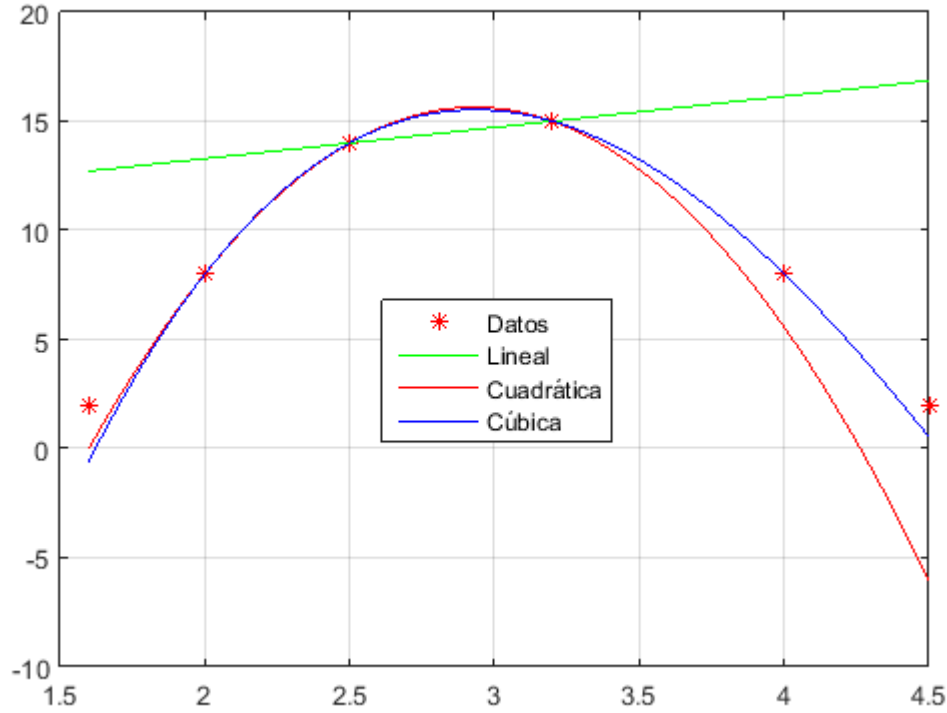


Figura 2: Interpolaciones de primer a tercer orden.

Si el grado del polinomio es 2, seleccionamos a 3 puntos para la interpolación, digamos (2, 8), (2.5, 14) y (3.2, 15); de esta manera el Polinomio ajustado según la fórmula de Lagrange es:

$$\begin{aligned}
 P_2(x) &= \frac{(x-2.5)(x-3.2)}{(2-2.5)(2-3.2)} \cdot 8 + \frac{(x-2)(x-3.2)}{(2.5-2)(2.5-3.2)} \cdot 14 + \frac{(x-2)(x-2.5)}{(3.2-2)(3.2-2.5)} \cdot 15 \\
 &= -\frac{185x^2}{21} + \frac{723x}{14} - \frac{1261}{21}
 \end{aligned}$$

Entonces $f(x) \approx P_2(x) = -\frac{185x^2}{21} + \frac{723x}{14} - \frac{1261}{21}$. Por tanto, $f(2.8) = 542/35 = 15.485714286$.

Si el grado del polinomio es 3, seleccionamos a 4 puntos para la interpolación, digamos (2, 8), (2.5, 14), (3.2, 15) y (4, 8); de esta manera el Polinomio ajustado según la fórmula de Lagrange es:

$$\begin{aligned}
 P_3(x) &= \frac{(x-2.5)(x-3.2)(x-4)}{(2-2.5)(2-3.2)(2-4)} \cdot 8 + \frac{(x-2)(x-3.2)(x-4)}{(2.5-2)(2.5-3.2)(2.5-4)} \cdot 14 + \\
 &\quad + \frac{(x-2)(x-2.5)(x-4)}{(3.2-2)(3.2-2.5)(3.2-4)} \cdot 15 + \frac{(x-2)(x-2.5)(x-3.2)}{(4-2)(4-2.5)(4-3.2)} \cdot 8 \\
 &= \frac{85x^3}{84} - \frac{2789x^2}{168} + \frac{5987x}{84} - \frac{1601}{21}
 \end{aligned}$$

Entonces $f(x) \approx P_3(x) = \frac{85x^3}{84} - \frac{2789x^2}{168} + \frac{5987x}{84} - \frac{1601}{21}$. Por tanto, $f(2.8) = 2693/175 = 15.388571429$.

1.1. Algoritmo para la Interpolación de Lagrange

A continuación se establece un Pseudocódigo para la interpolación de Lagrange. Este algoritmo se establece para calcular una sola predicción de grado n -ésimo, donde $n + 1$ es el número de puntos asociados con datos.

```
1: function LAGRANGE(x,y,n,xx)
2:   sumx=0
3:   for i=1:n+1 do
4:     product=yi
5:     for j=1:n+1 do
6:       if i≠j then
7:         product=product*(xx-xj)/(xi - xj)
8:       end if
9:     end for
10:    sum=sum+product
11:  end for
12:  Lagrange=sum
13: end function
```

2. Método de Newton

Aunque el método de Lagrange es conceptualmente simple, no se presta a un algoritmo eficiente. Se obtiene un mejor procedimiento computacional con el método de Newton, donde el polinomio de interpolación se escribe en la forma

$$P_{n-1} = a_1 + (x - x_1)a_2 + (x - x_1)(x - x_2)a_3 + \cdots + (x - x_1)(x - x_2) \cdots (x - x_{n-1})a_n$$

2.1. Evaluación del polinomio

Este polinomio se presta a un procedimiento de evaluación eficiente. Considerar por ejemplo, cuatro puntos de datos ($n = 4$). Aquí el polinomio de interpolación es

$$P_3 = a_1 + (x - x_1)a_2 + (x - x_1)(x - x_2)a_3 + (x - x_1)(x - x_2)(x - x_3)a_4$$

que puede ser evaluado mediante las siguientes relaciones de recurrencia:

$$\begin{aligned}P_0(x) &= a_4 \\P_1(x) &= a_3 + (x - x_3)P_0(x) \\P_2(x) &= a_2 + (x - x_2)P_1(x) \\P_3(x) &= a_1 + (x - x_1)P_2(x)\end{aligned}$$

Para un n arbitrario tendremos

$$P_0(x) = a_n, \quad P_k(x) = a_{n-k} + (x - x_{n-k})P_{k-1}(x), \quad k = 1, 2, \dots, n-1$$

Denotando el conjunto de coordenadas x de los puntos de datos por xData, y el número de puntos de datos por n , tenemos el siguiente algoritmo para calcular $P_{n-1}(x)$:

```
1 function p = newtonPoly(a, xData, x)
2 % Retorna el valor del polinomio de Newton en x.
```

```

3 %USAGE: p = newtonPoly(a,xData,x)
4 %a = arreglo de coeficientes del polinomio;
5 %debe de ser calculado antes usando newtonCoeff.
6 %xData = x-coordenadas de los puntos de datos.
7 n = length(xData);
8 p = a(n);
9 for k = 1:n-1;
10     p = a(n-k) + (x - xData(n-k))*p;
11 end

```

2.2. Cálculo de Coeficientes

Los coeficientes de $P_{n-1}(x)$ son determinados forzando al polinomio a pasar por cada punto: $y_i = P_{n-1}(x_i), i = 1, 2, \dots, n$. Esto lleva a las ecuaciones simultaneas:

$$\begin{aligned}
 y_1 &= a_1 \\
 y_2 &= a_2 + (x_2 - x_1)a_2 \\
 y_3 &= a_2 + (x_3 - x_1)a_2 + (x_3 - x_1)(x_3 - x_2)a_3 \\
 &\vdots \\
 y_n &= a_1 + (x_n - x_1)a_2 + (x_n - x_1)(x_n - x_2) \cdots (x_n - x_{n-1})a_n
 \end{aligned}$$

Introduciendo las *diferencias divididas*

$$\begin{aligned}
 \nabla y_i &= \frac{y_i - y_1}{x_i - x_1}, i = 2, 3, \dots, n \\
 \nabla^2 y_i &= \frac{\nabla y_i - \nabla y_2}{x_i - x_2}, i = 3, 4, \dots, n \\
 \nabla^3 y_i &= \frac{\nabla^2 y_i - \nabla^2 y_3}{x_i - x_3}, i = 4, 5, \dots, n \\
 &\vdots \\
 \nabla^{n-1} y_n &= \frac{\nabla^{n-2} y_n - \nabla^{n-2} y_{n-1}}{x_n - x_{n-1}}
 \end{aligned}$$

la solución del sistema es:

$$a_1 = y_1, \quad a_2 = \nabla y_2, \quad a_3 = \nabla^2 y_3, \quad \dots \quad a_n = \nabla^{n-1} y_n$$

Si los coeficientes son calculados a mano, es conveniente trabajar en el formato de la tabla:

x_1	y_1				
x_2	y_2	∇y_2			
x_3	y_3	∇y_3	$\nabla^2 y_3$		
x_4	y_4	∇y_4	$\nabla^2 y_4$	$\nabla^3 y_4$	
x_5	y_5	∇y_5	$\nabla^2 y_5$	$\nabla^3 y_5$	$\nabla^4 y_5$

Cuadro 1: Diferencias finitas divididas.

Los términos en la diagonal $(y_1, \nabla y_2, \nabla^2 y_3, \nabla^3 y_4, \nabla^4 y_5)$ en la tabla son los coeficientes del polinomio. Si los puntos son ordenados en otro orden, las entradas de la tabla cambiarán, pero el

polinomio resultante sera el mismo.

Los cálculos de la máquina se realizan mejor dentro de una matriz unidimensional empleando el siguiente algoritmo:

```

1 function a = newtonCoeff(xData,yData)
2 % Retorna los coeficientes del polinomio de Newton.
3 % USAGE: a = newtonCoeff(xData,yData)
4 % xData = x-coordenadas de los puntos de datos.
5 % yData = y-coordenadas de los puntos de datos.
6
7 n = length(xData);
8 a = yData;
9 for k = 2:n
10     a(k:n) = (a(k:n) - a(k-1))./(xData(k:n) - xData(k-1));
11 end

```

Inicialmente, a contiene los valores y de los datos, de modo que es idéntico a la segunda columna del Cuadro 1. Cada pasada a través de for-loop genera las entradas en la columna siguiente, que sobrescriben los elementos correspondientes de a . Por lo tanto, al final contiene los términos diagonales del Cuadro 1; es decir, los coeficientes del polinomio.

Ejemplo 2.1. Los puntos en la tabla yacen en la gráfica de $f(x) = 4.8 \cos \pi x/20$. Interpolar estos datos por el método de Newton en $x = 0, 0.5, 1.0, \dots, 8.0$ y compare los resultados con los valores “exactos” dados por $y = f(x)$.

x	0.15	2.30	3.15	4.85	6.25	7.95
y	4.79867	4.49013	4.2243	3.47313	2.66674	1.51909

```

1 %Ejemplo 1.1 (Interpolacion de Newton)
2 xData = [0.15; 2.3; 3.15; 4.85; 6.25; 7.95];
3 yData = [4.79867; 4.49013; 4.22430; 3.47313; 2.66674; 1.51909];
4 a = newtonCoeff(xData,yData);
5 '      x      yInterp      yExact '
6 for x = 0:0.5:8
7     y = newtonPoly(a,xData,x);
8     yExact = 4.8*cos(pi*x/20);
9     fprintf(' %10.5f ',x,y,yExact)
10    fprintf('\n')
11 end

```

Los resultados son:

```

1 ans =
2
3      x      yInterp      yExact
4
5      0.00000      4.80003      4.80000
6      0.50000      4.78518      4.78520
7      1.00000      4.74088      4.74090
8      1.50000      4.66736      4.66738
9      2.00000      4.56507      4.56507

```

10	2.50000	4.43462	4.43462
11	3.00000	4.27683	4.27683
12	3.50000	4.09267	4.09267
13	4.00000	3.88327	3.88328
14	4.50000	3.64994	3.64995
15	5.00000	3.39411	3.39411
16	5.50000	3.11735	3.11735
17	6.00000	2.82137	2.82137
18	6.50000	2.50799	2.50799
19	7.00000	2.17915	2.17915
20	7.50000	1.83687	1.83688
21	8.00000	1.48329	1.48328

3. Splines Cúbicos

El objetivo en los trazadores cúbicos es obtener un polinomio de tercer grado para cada intervalo entre los nodos:

$$f_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i, \quad (1)$$

Así, para $n+1$ datos ($i = 0, 1, 2, \dots, n$), existen n intervalos y, en consecuencia, $4n$ incógnitas a evaluar. Como con los trazadores cuadráticos, se requieren $4n$ condiciones para evaluar las incógnitas. Éstas son:

1. Los valores de la función deben ser iguales en los nodos interiores ($2n - 2$ condiciones).
2. La primera y última función deben pasar a través de los puntos extremos (2 condiciones).
3. Las primeras derivadas en los nodos interiores deben ser iguales ($n - 1$ condiciones).
4. Las segundas derivadas en los nodos interiores deben ser iguales ($n - 1$ condiciones).
5. Las segundas derivadas en los nodos extremos son cero (2 condiciones).

Los cinco tipos de condiciones anteriores proporcionan el total de las $4n$ ecuaciones requeridas para encontrar los $4n$ coeficientes. Mientras es posible desarrollar trazadores cúbicos de esta forma, presentaremos una técnica alternativa que requiere la solución de sólo $n - 1$ ecuaciones. Aunque la obtención de este método es un poco menos directo que el de los trazadores cuadráticos, la ganancia en eficiencia bien vale la pena.

La ecuación cúbica en cada intervalo:

$$\begin{aligned}
f_i(x) = & \frac{f_i''(x_{i-1})}{6(x_i - x_{i-1})}(x_i - x)^3 + \frac{f_i''(x_i)}{6(x_i - x_{i-1})}(x - x_{i-1})^3 \\
& + \left[\frac{f(x_{i-1})}{x_i - x_{i-1}} - \frac{f''(x_{i-1})(x_i - x_{i-1})}{6} \right] (x_i - x) \\
& + \left[\frac{f(x_i)}{x_i - x_{i-1}} - \frac{f''(x_i)(x_i - x_{i-1})}{6} \right] (x - x_{i-1})
\end{aligned} \quad (2)$$

Esta ecuación contiene sólo dos incógnitas (las segundas derivadas en los extremos de cada intervalo). Las incógnitas se evalúan empleando la siguiente ecuación:

$$(x_i - x_{i-1})f''(x_{i-1}) + 2(x_{i+1} - x_{i-1})f''(x_i) + (x_{i+1} - x_i)f''(x_{i+1}) = \frac{6}{x_{i+1} - x_i} [f(x_{i+1}) - f(x_i)] + \frac{6}{x_i - x_{i-1}} [f(x_{i-1}) - f(x_i)] \quad (3)$$

Si se escribe esta ecuación para todos los nodos interiores, resultan $n - 1$ ecuaciones simultáneas con $n - 1$ incógnitas. (Recuerde que las segundas derivadas en los nodos extremos son cero.)

3.1. Cálculo de curvaturas

La primera etapa de la interpolación por splines cúbicos es tomar las ecuaciones de (3) y resolverlas para las $f''(x_i)$ desconocidas (recordemos que $f''(x_1) = f''(x_n) = 0$). Esta tarea se lleva a cabo mediante la función:

```

1 function k = splineCurv(xData,yData)
2 % Retorna las curvaturas de los splines cubicos en los nudos.
3 %USAGE: k = splineCurv(xData,yData)
4 %xData = x-coordenadas de puntos de datos.
5 %yData = y-coordenadas de puntos de datos.
6
7 n = length(xData);
8 c = zeros(n-1,1); d = ones(n,1);
9 e = zeros(n-1,1); k = zeros(n,1);
10 c(1:n-2) = xData(1:n-2) - xData(2:n-1);
11 d(2:n-1) = 2*(xData(1:n-2) - xData(3:n));
12 e(2:n-1) = xData(2:n-1) - xData(3:n);
13 k(2:n-1) = 6*(yData(1:n-2) - yData(2:n-1))./(xData(1:n-2) - xData
        (2:n-1)) - 6*(yData(2:n-1) - yData(3:n))./(xData(2:n-1) - xData
        (3:n));
14 [c,d,e] = LUdec3(c,d,e);
15 k = LUsol3(c,d,e,k);

```

3.2. Evaluación

La función splineEval calcula el interpolador en x de la ecuación (2). La subfunción findSeg encuentra el segmento del spline que contiene x por el método de bisección. Devuelve el número de segmento; es decir, el valor del subíndice i en la ecuación (2).

```

1 function y = splineEval(xData,yData,k,x)
2 % Retorna el valor de la interpolacion por splines cubico en x.
3 %USAGE: y = splineEval(xData,yData,k,x)
4 %xData = x-coordenadas de puntos de datos.
5 %yData = y-coordenadas de puntos de datos.
6 %k = curvaturas del spline en los nodos;
7 %retornados por la funcion splineCurv.
8
9 i = findSeg(xData,x);

```



```

10 h = xData(i) - xData(i+1);
11 y = ((x - xData(i+1))^3/h - (x - xData(i+1))*h)*k(i)/6.0 - ((x -
    xData(i))^3/h - (x - xData(i))*h)*k(i+1)/6.0 + yData(i)*(x -
    xData(i+1))/h - yData(i+1)*(x - xData(i))/h;
12
13 function i = findSeg(xData,x)
14 % Retorna el segmento indexado que contiene x.
15 iLeft = 1; iRight = length(xData);
16 while 1
17     if(iRight - iLeft) <= 1
18         i = iLeft; return
19     end
20     i = fix((iLeft + iRight)/2);
21     if x < xData(i)
22         iRight = i;
23     else
24         iLeft = i;
25     end
26 end

```

Ejemplo 3.1. Obtenga trazadores cúbicos para los datos

x	2	3	5	7
$f(x)$	6	19	99	291

- Pronostique $f(2)$ y $f(2.5)$.
- Verifique $f_1(3)$ y $f_2(3)$ son iguales a 19.

Una vez realizadas las funciones que permiten realizar interpolación por splines cúbicos solo es cuestión de realizar los siguientes cálculos:

```

1 xData = [2;3;5;7];
2 yData = [6;19;99;291];
3 k = splineCurv(xData,yData);
4 %Para evaluar f(4)
5 splineEval(xData,yData,k,4)
6 %Para evaluar f(2.5)
7 splineEval(xData,yData,k,2.5)

```

Podemos de hecho encontrar el polinomio cúbico en cada intervalo, una vez conocidos los valores de las segundas derivadas (el vector k), tenemos que:

$$f''_0(x_0) = 0, f''_1(x_1) = 1.67908745247148, f''_2(x_2) = -1.53307984790875 \text{ y } f''_3(x_3) = 0$$

Sustituyendo en la ecuación (2) tendremos que:

$$f_1(x) = \frac{f''_1(x_1)}{6(x_1 - x_0)^3}(x - x_0)^3 + \left[\frac{f(x_0)}{x_1 - x_0} \right] (x_1 - x) + \left[\frac{f(x_1)}{x_1 - x_0} - \frac{f''_1(x_1)(x_1 - x_0)}{6} \right] (x - x_0)$$

$$f_1(x) = \frac{26x^3}{11} - \frac{156x^2}{11} + 39x - \frac{376}{11}$$

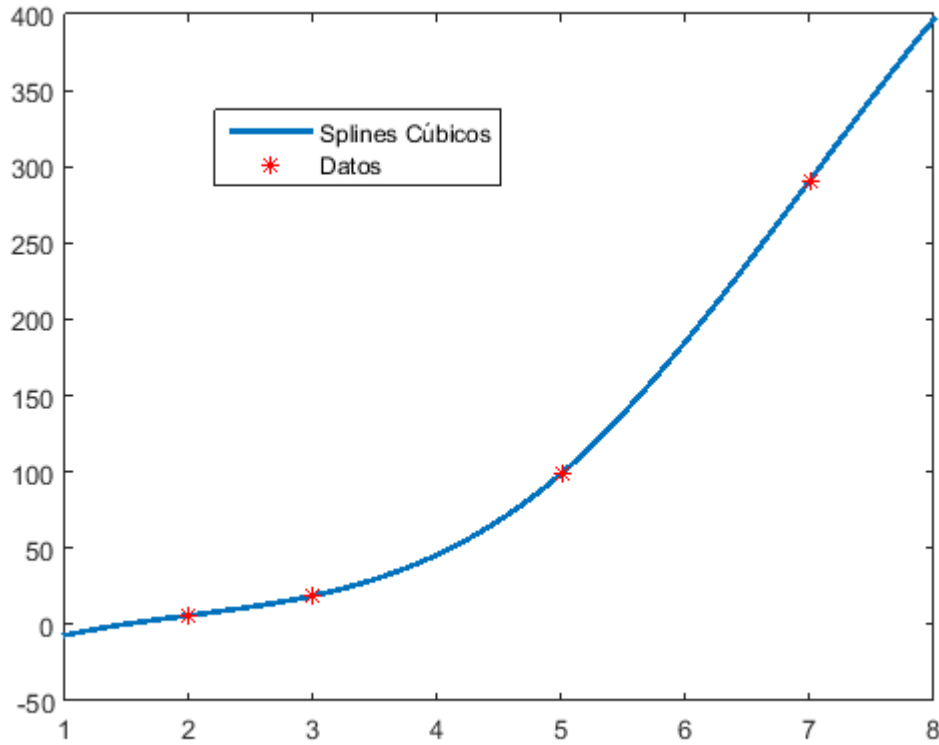


Figura 3: Splines Cúbicos.

$$f_2(x) = \frac{f_2''(x_1)}{6(x_2 - x_1)}(x_2 - x)^3 + \frac{f_2''(x_2)}{6(x_2 - x_1)}(x - x_1)^3$$

$$+ \left[\frac{f(x_1)}{x_2 - x_1} - \frac{f''(x_1)(x_2 - x_1)}{6} \right] (x_2 - x) + \left[\frac{f(x_2)}{x_2 - x_1} - \frac{f''(x_2)(x_2 - x_1)}{6} \right] (x - x_1)$$

$$f_2(x) = \frac{89x^3}{44} - \frac{489x^2}{44} + \frac{1311x}{44} - \frac{1099}{44}$$

Y es fácil comprobar que $f_1(3) = f_2(3) = 19$.

Problemas Propuestos

Problema 3.1. Los siguientes datos provienen de una tabla y fueron medidos con alta precisión. Use el mejor método numérico (para este tipo de problema) para determinar y en $x = 3.5$. Observe que un polinomio dará un valor exacto. Su solución debe probar que su resultado es exacto.

x	0	1.8	5	6	8.2	9.2	12
y	26	16.415	5.375	3.5	2.015	2.54	8

Problema 3.2. Utilice interpolación inversa para determinar el valor de x que corresponde a $f(x) = 0.85$, para los datos tabulados siguientes:

x	0	1	2	3	4	5
$f(x)$	0	0.5	0.8	0.9	0.941176	0.961538

Observe que los valores de la tabla se generaron con la función $f(x) = \frac{x^2}{1+x^2}$.

- Determine en forma analítica el valor correcto.
- Use interpolación cúbica de x versus y .
- Utilice interpolación inversa con interpolación cuadrática y la fórmula cuadrática.
- Emplee interpolación inversa con interpolación cúbica y bisección. Para los incisos b) a d) calcule el error relativo porcentual verdadero.

Problema 3.3. Los puntos

x	-2	1	4	-1	3	-4
y	-1	2	59	4	24	-53

yacen en un polinomio. Use la tabla de diferencias divididas del Método de Newton para determinar el grado del polinomio.

Problema 3.4. Genere ocho puntos igualmente espaciados de la función

$$f(t) = \sin^2 t$$

de $t = 0$ a 2π . Ajuste estos datos con a) un polinomio de interpolación de séptimo grado y b) un trazador cúbico.

Problema 3.5. La función de Runge se escribe como

$$f(x) = \frac{1}{1+25x^2}.$$

- Genere y grafique el polinomio de interpolación de Lagrange usando valores de la función equiespaciados correspondientes a $x = -1, -0.5, 0, 0.5, 1$
- Use los cinco puntos de a) para estimar $f(0.8)$ con polinomios de interpolación de Newton de grado primero a cuarto.
- Genere un trazador cubico usando los cinco puntos de a).

Problema 3.6. Una aplicación útil de la interpolación de Lagrange se denomina búsqueda en la tabla. Como el nombre lo indica, involucra “buscar” un valor intermedio en una tabla. Para desarrollar dicho algoritmo, en primer lugar se almacena la tabla de los valores de x y $f(x)$ en un par de arreglos unidimensionales. Después, dichos valores se pasan a una función junto con el valor de x que se desea evaluar. La función hace luego dos tareas. En primer lugar, hace un ciclo hacia abajo de la tabla hasta que encuentra el intervalo en el que se localiza la incógnita. Después aplica una técnica como la interpolación de Lagrange para determinar el valor apropiado de $f(x)$. Desarrolle una función así con el uso de un polinomio cúbico de Lagrange para ejecutar la interpolación. Para intervalos intermedios ésta es una buena elección porque la incógnita se localiza en el intervalo a la mitad de los cuatro puntos necesarios para generar la expresión cúbica. Para los intervalos primero y último, use un polinomio cuadrático de Lagrange. Asimismo, haga que el código detecte cuando el usuario pida un valor fuera del rango de las x . Para esos casos, la función debe desplegar un mensaje de error. Pruebe su programa para $f(x) = \ln x$ con los datos $x = 1, 2, \dots, 10$.